Neuroinformatics: Modeling Neurons Spiking Responses to External Stimuli

Ali Al-Dulaimi
Matr. #: 1019322
aaldulaimi@uni-osnabrueck.de

Outline

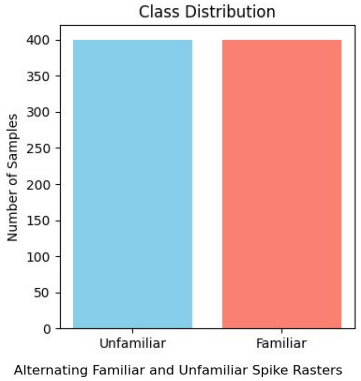
- Data Visualization
- Neural Coding Methods
- Metrics
- Results
- Next Steps

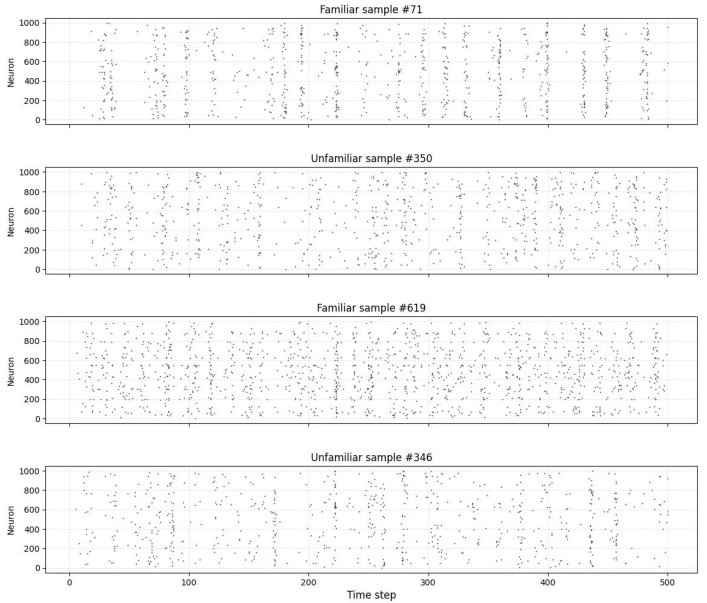
1. Data Visualization

We first visualized the **class distribution** using a bar chart, confirming a **balanced dataset** with equal numbers of **familiar** and **unfamiliar** stimuli (400 each).

To explore raw neural responses, we plotted **raster plots** for 4 selected neurons across trials. These show **spike timings** for each neuron when presented with either familiar or unfamiliar stimuli.

The raster plots are **alternatingly stacked**, allowing us to visually compare neural firing patterns between the two stimulus types. This helps highlight differences in **spike timing** and **rate of firing** across conditions.





2. Neural Coding Methods

- Rate coding
- Sparsity coding
- Frequency coding

2.1. Rate Coding

Firing Rate Coding (how much neurons fire) => Average number of spikes per neuron over time, Very simple, very biologically meaningful, easy for Logistic Regression or any classifier

- Represents **how often each neuron fires** over a time window.
- Computed as the **average number of spikes per neuron** across the stimulus duration.
- **Biologically inspired**: closely mirrors how neurons encode stimulus intensity.
- Produces simple, continuous features that work well with **logistic regression** and other classical classifiers. Serves as a strong **baseline encoding method** due to its interpretability and efficiency.

2.2. Sparsity Coding

Sparsity Coding (how many neurons are involved) => Fraction of neurons that fired at least once, Captures "distributed vs. sparse" population coding. Familiar stimuli may activate more neurons in a coordinated way, whereas unfamiliar stimuli might result in sparse or disorganized firing. Sparsity features also showed a large KL divergence between familiar and unfamiliar classes, suggesting strong discriminative power for classification.

- Measures **how few neurons** are active in response to a stimulus.
- For each trial, we compute a **single number** representing the **proportion of inactive neurons**.
- Low dimensionality — results in only **one feature per sample**.
- Useful as a **compressed summary** of neural activity, but may lose a lot of information captured in the temporal data (timesteps).

2.3. Frequency Coding

Frequency Coding (how spikes are distributed across time patterns (bursty or rhythmic)) => (Fourier Transform) How much "power" is in low-frequency components of spike trains, Captures possible rhythmic or oscillatory neural patterns; performed very well in classification (85% accuracy!) Frequency domain features can reveal temporal structures such as bursts or rhythmic firing, which might differ between familiar and unfamiliar stimuli. Frequency coding provided excellent classification performance (85% accuracy), confirming its relevance for this task.

- Captures **how often** each neuron fires, treating spikes as **oscillatory signals**.
- For each neuron, we compute the **dominant frequency** using the **Fast Fourier Transform (FFT)**.
- Preserves **temporal firing patterns** that simple rate coding might miss.
- Can expose the **periodic firing behavior** related to stimulus familiarity.
- Produces quality, high-dimensional features that work well with **nonlinear models** like SVM or MLP.

In **rate coding**, we collapse over the time axis by **averaging spike activity per neuron**, keeping all neurons as features. In **sparsity coding**, we first collapse over time by checking if a neuron spiked at all, and then collapse across neurons **to compute the fraction of active neurons for each sample**. In **frequency coding**, we collapse over the time axis by applying a Fourier transform to each neuron spike train, and then** summarize the low-frequency components by summing their magnitudes**. We keep all neurons as features, with each feature reflecting the strength of slow oscillations in that neuron's firing activity.

Reasons I chose these methods:

- They are fast to compute and easy to interpret.
- Easy to implement.
- Easy to interpret

3. Metrics

- KL Divergence
- Accuracy
- F1 Score
  - Precision
  - Recall

3.1. KL Divergence

KL Divergence (Exploratory Metric)

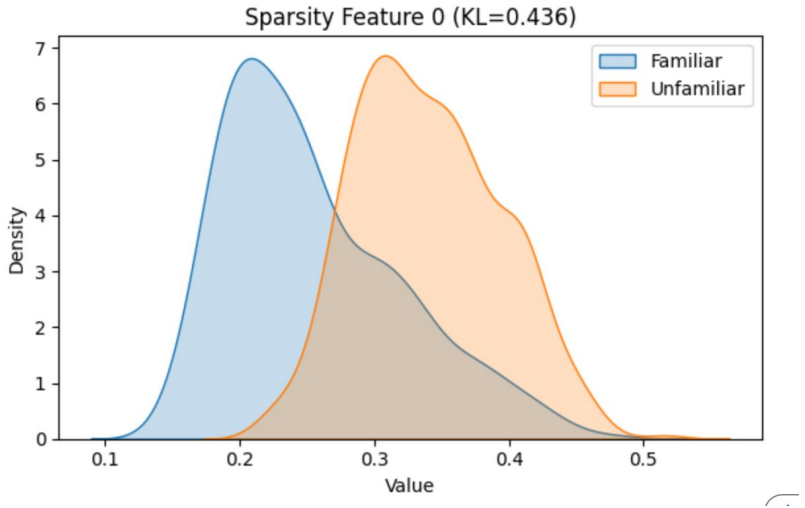
- Used to **measure the difference** between the distributions of familiar vs unfamiliar responses.
- Applied separately to features extracted via:
    - **Firing Rate Coding**: moderate divergence — distributions overlap, but separable.
    - **Sparsity Coding**: low divergence — distributions are compressed (because we compress two dimensions into a single value!) and harder to distinguish.
    - **Frequency Coding**: high divergence — distributions show clear separation, indicating strong class information.
- Visualized using **histograms or KDE plots** per feature, grouped by class (we picked KDE in our notebook).
- Helped us identify the **most informative neurons** for feature selection.

3.1.1 KDE Plots

To visualize class separability, we used **Kernel Density Estimation (KDE) plots**, which show smooth approximations of feature distributions for **familiar vs unfamiliar** stimuli.

KDE plots made it easy to **see where the distributions overlap** and where they differ. Greater separation in KDE curves indicates a more informative feature — which aligned well with **KL divergence values**.

```
Class balance: 400 familiar, 400 unfamiliar
Average KL Divergence: 0.4361
Top 3 informative features (by KL): [0] with values [0.43609842]
```



Classification Metrics

Accuracy: Overall percentage of correctly classified samples.

- Easy to interpret but may be misleading if classes are imbalanced (not a concern here).

F1-score: Harmonic mean of **precision** and **recall**.

- More robust when **false positives and false negatives** carry different consequences.
- Better reflects model quality in tasks with potential subtle decision boundaries.

To view both of the metrics in a compact way we use sklearn classification report for binary classification tasks:

```
=== Evaluation Report: Naive Bayes ===
Accuracy : 0.7250
F1-score : 0.6944

Detailed Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.74      0.75        89
           1       0.68      0.70      0.69        71

    accuracy                           0.72       160
   macro avg       0.72      0.72      0.72       160
weighted avg       0.73      0.72      0.73       160
```

This shows the initial classification report including F1 score, accuracy, precision and recall of a Naive Bayes baseline model

4.1. Results

Performance Improvement Strategies I used in my notebook (in this PDF, I will only show the best result and briefly compare the models):

1. Train Naive Bayes on:
    - X_rate
    - X_sparsity
    - **X_frequency**
2. Train multiple classification models on the best performing features:
    - Logistic Regression
    - Random Forests
    - **MLB**
    - **SVM**
3. Train best performing model on a combination of features:
    - np.hstack([X_rate, X_sparsity, X_frequency])
4. Train best performing model, best performing features set:
    - Selecting neuron's with highest information
5. **Dimensionality reduction (PCA) on all X_frequency neurons and then best performing model**

4.2. Results

1. Neural Coding vs Naive Bayes (Baseline)

- **Rate Coding**: Accuracy = **72.5%**, F1-score = *0.69*
- **Sparsity Coding**: Accuracy = **71.2%**, F1-score = *0.68*
- **Frequency Coding**: Accuracy = **80.6%**, F1-score = *0.78*

> **Frequency coding** provided the most informative representation for classifying familiar & unfamiliar stimuli.

4.3. Classifier Comparison (on Frequency Features)

Naive Bayes
Accuracy: 80.6%
F1-score: 78%

Logistic Regression
Accuracy: 85.0%
F1-score: 84%

Random Forest
Accuracy: 87.5%
F1-score: 86%

MLP Classifier
Accuracy: 88.8%
F1-score: 87%

XGBoost
Accuracy: 83.8%
F1-score: 81%

SVM (Linear Kernel)
Accuracy: 91.3%
F1-score: 90%

> **SVM and MLP** achieved the best overall results using the full frequency-coded feature set.

With feature selection, top-K neurons (KL divergence ranking) performance peaked at K=20. Adding more neurons introduced redundant or noisy features that degraded performance.

4.4.4. Best Result!

Dimensionality Reduction: PCA + SVM

- Original feature dimensions: 1000
- PCA-reduced dimensions: 431 (retaining 95% variance)

SVM (PCA features) — Accuracy: **93.1%**, F1-score: **92%**

PCA compression **improved generalization**, suggesting the most meaningful neural variation is **low-dimensional**.

Combining **frequency coding** with **PCA** and a nonlinear classifier (SVM) achieved the highest performance.

The data is rich in temporal patterns, but much of its structure can be captured using **fewer, well-chosen features** (using FFT or alike)

5. Next Steps

1. Most important step to take is to find a richer neural coding method, rate coding only worked as a baseline. While frequency coding performance gave us confidence to explore even confidence in trying new, more advance neural coding methods, ones that respect the relational/temporal information like **interspike interval coding**, **synchrony-based features**, or **temporal convolution.**
2. **Testing the generalizability of the model by feeding it data from different neural regions**
3. Collect more trials or include more stimuli to help models generalize better (more data -> more params that are learnable). A larger dataset can support **deeper (more complex) models**, **better feature selection**, and **more confident performance evaluation**.